

Running Head: AI AGENT DEVELOPMENT

AI AGENT DEVELOPMENT

Aksel Dindisyan, 32109

August, 2 2025 – September, 15 2025

Orbislabs.ai

Mr. Kamer Kaya

September, 30 2025

Sabanci University

Faculty of Engineering and Natural Sciences

Computer Science & Engineering

ABSTRACT

This internship was done in Orbislabs which is a startup that develops agentic solutions. My assignment was developing AI agents for the company, and I had two main projects, which were the CV Analyzer Agent and the Text2SQL Agent. These projects were developed for our customer companies that want to automate some parts of their workflows like analysing CVs and writing SQL queries. For the CV Analyzer agent, two main solutions were developed, which were an SBERT-based solution and an RAG based solution. For the Text2SQL Agent, also two main solutions were developed, which were a Vanna AI based solution and another open source model developed by LangChain. Both projects were finished by me or my teammates at the end of my internship period. As an advice, I would recommend taking Large Language Models: Theoretical Foundations and Practical Applications (CS 455) to know more about AI agents and I would suggest new interns to work in startups to experience a different type of work place.

TABLE OF CONTENTS

1. INTRODUCTION.....	4
2. COMPANY BACKGROUND	4
3. PROJECT DESCRIPTION and ANALYSIS	5
3.1 CV ANALYZER AGENT	5
3.1.1 PROJECT PURPOSE.....	5
3.1.2 PROJECT DESCRIPTION	5
3.1.3 TECHNICAL ASPECTS	6
3.1.4 METHODOLOGY and TOOLS.....	7
3.1.5 PROJECT RESULTS	7
3.1.6 LITERATURE REVIEW	8
3.2 TEXT2SQL AGENT	8
3.2.1 PROJECT PURPOSE.....	8
3.2.2 PROJECT DESCRIPTION	9
3.2.3 TECHNICAL ASPECTS	9
3.2.4 METHODOLOGY and TOOLS.....	11
3.2.5 PROJECT RESULTS	12
3.2.6 LITERATURE REVIEW	12
4. INTERNSHIP EXPERIENCE.....	12
4.1 RELATION to UNDERGRADUATE CURRICULLUM	12
4.2 DIFFICULTIES.....	13
4.3 NEW SKILLS	13
5. CONCLUSIONS	13
6. RECOMMENDATIONS	14

1. INTRODUCTION

In this paper, I will talk about CV Analyzer Agent and Text2SQL Agent that I have developed in Orbislabs during my internship. In recent decades hiring process has become more complicated due to increasing number of applications. In order to make hiring process faster and more efficient, I developed a CV Analyzer agent that gets CV PDFs and job description as an input and outputs the most qualified CVs from the candidate pool. Also, I worked on the development of a Text2SQL Agent which converts natural language queries to SQL queries. This can help high level managers interact with databases without any SQL knowledge and makes query writing process a lot faster. In the following pages of this report these agents will be discussed in more details. Firstly, there will be some background information about Orbislabs. Secondly, it will continue with description and analysis of the internship such as the purpose of the project, things that I have done and the results. Thirdly, I will share my observations and analysis on the internship by giving advice to future interns and finally my paper will be concluded with a conclusion which will summarize the important points of the paper and discuss the results of the project.

2. COMPANY BACKGROUND

Orbislabs is a small startup which was founded in 2025. It is also a part of the KOMTAS ecosystem. KOMTAS is a bigger umbrella company which consists of various data analytics and AI companies. Orbislabs mainly focuses on developing AI agentic solutions to business in order to make business more efficient. They develop task assistants, multi-modal agents, autonomous agents. They research new LLM and AI agent technologies and implement them in their solutions. Due to its size, Orbislabs doesn't have separate departments, so I didn't have a specific department. I was responsible of creating AI agents. The main projects that I worked on are CV analyser, Text2SQL agent.

3. PROJECT DESCRIPTION and ANALYSIS

3.1 CV ANALYZER AGENT

3.1.1 PROJECT PURPOSE

Our company needed to create an AI agent to automate the candidate selection for a job opening for one of our customer companies. The customer wanted this agent in order to select candidates much faster and more efficiently. Also, by automating the candidate selection, Human Resources can focus on more important tasks. Otherwise, Human Resources personnel would need to read hundreds or even thousands of CVs, which is inefficient and time consuming. Moreover, by automating job selection, we also reduce human biases that can affect the hiring process.

3.1.2 PROJECT DESCRIPTION

The CV Analyzer gets CV PDFs and job description as input and produces an output which is a list of the best candidates with their matching and missing qualities from the job description. Also, if there is no exact matching candidate, it should output similar ones. Moreover, users should be able to chat with the agent in order to ask further questions about the CV database.

3.1.3 TECHNICAL ASPECTS

At first, a method based on sentence embedding was used. PDFs were converted into texts by using PyMuPDF python library. Then those texts and job description were given as input to the SBERT model. SBERT is a special BERT model that embeds sentences instead of words. In our example, SBERT embeds sentences from both job description and CVs. Then it matches the similar sentences. Points are assigned to CVs based on the number of matches between a CV and a job description. CVs with higher points are more correlated with the job description. The system also outputs which sentences were matched in order to be more transparent in the decision-making progress. This was scrapped due to platform limitations. Orbislabs develops AI agents on a modified version of a langflow, which they host in a remote server. Normally, I used this method, the local computer CPU was used for inference however in order to use the exact same method at remote platform we need to rent a CPU separately and or use the already rented CPU for also SBERT inference.

Secondly, an RAG based agent was employed. For converting PDFs, the exact code and the library from the previous method were used. Then CVs that were converted to texts were divided into chunks, and the chunks were fed into an FAISS vector database by using the Cohere embedding model. Then FAISS component outputs the 5 most correlated CVs, after we give a job description as a chat input. Then we send those CVs to a Cohere Rerank component in order to make the ranking more robust. After that, the final result is appended to the system prompt of the Cohere LLM, which will do the talking. We also feed the chat input to the LLM model with a system prompt. Based on the inputs LLM model outputs the list of selected candidate names and important qualities of each candidate. Later on we also added a memory component to the system prompt in order to ask questions about previous outputs of the LLM model.

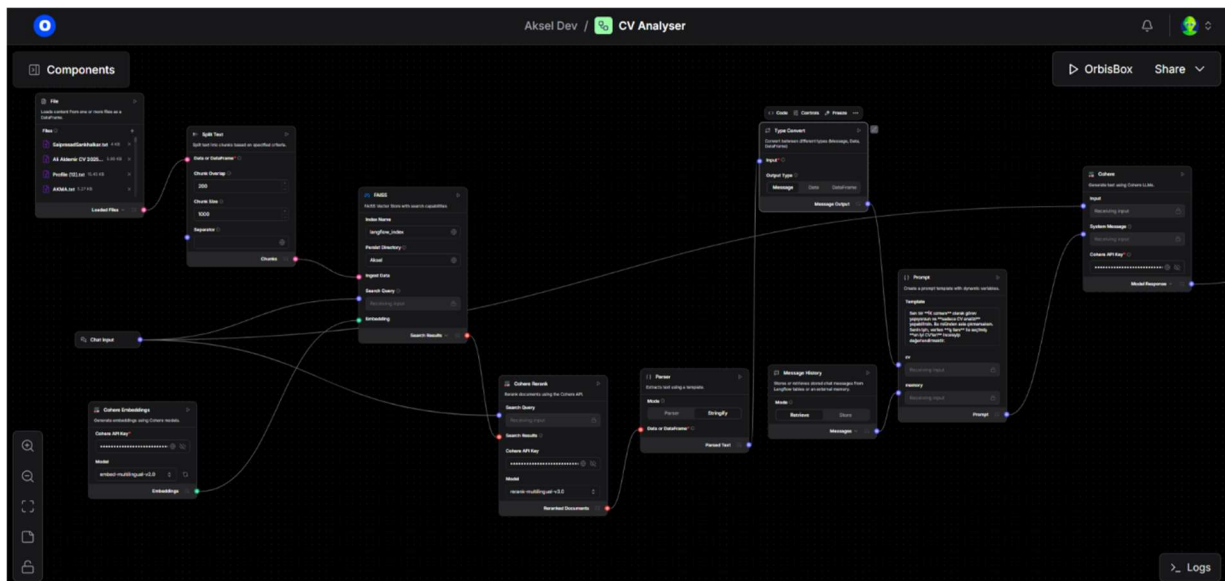


Figure 1: CV Analyzer Components

3.1.4 METHODOLOGY and TOOLS

There were several tools that were used in the development. For converting text to PDF, PyMuPDF python library was used. For the first attempt, SBERT was used to embed the text. On the other hand, Cohere Embedding and FAISS vector database were used to embed the text and store them meaningfully in the second attempt. Cohere Rerank was also used in order to have a more robust ranking, and Cohere LLM were used to talk with the Agent about the results. Lastly, the whole second attempt was produced on OrbisFlow(a modified version of Langflow).

3.1.5 PROJECT RESULTS

The first method was unsuccessful due to platform limitations, but on the other hand second method was successful. I was working on this project with other interns who also developed their own agents for the same problem. At the end, we merged our designs into one agent, which is very similar to the one that I designed, except it uses a hierarchical chunking model and it doesn't use a reranking model. This final form was tested by the customer and

approved by them with some minor feedback. The remaining part of the project was given to another intern, and I was assigned to Text2SQL project at this point.

3.1.6 LITERATURE REVIEW

During the CV Analyser project, I was also assigned to research machine learning and NLP solutions for CV analysing problem. I used Sudha's "Personality Prediction Through CV Analysis using Machine Learning Algorithms for Automated E-Recruitment Process" (Sudha et al., 2021) and Singh's "Personality prediction through CV analysis using machine learning techniques" (Singh et al., 2023) in order to study methods to guess personality of the candidates based on their CV. Also, I researched what kind of techniques were used to select candidates for job openings and university applications. For this job I used Haddad & Mercier-Laurent's "Curriculum Vitae (CVs) Evaluation Using Machine Learning Approach" (Haddad & Mercier-Laurent, 2021), Gupta's "Resume Analysis through Machine Learning" (Gupta et al., 2025), Onukwugha's "Resume Optimization Model Using Machine Learning Techniques" (Onukwugha et al., 2024) and Bhoir's "Resume Parser using hybrid approach to enhance the efficiency of Automated Recruitment Processes" (Bhoir et al., 2023).

3.2 TEXT2SQL AGENT

3.2.1 PROJECT PURPOSE

An insurance company with a huge dataset wanted us to create a Text2SQL AI Agent in order to convert natural language queries to SQL queries. This will allow high level managers who don't know SQL to ask questions directly to the database without needing the support of the data analytics team. By doing that, it will make getting the necessary data faster and more efficient. Also, it will reduce the responsibilities of the data analytics team, which

can focus on more important issues rather than consuming time to retrieve specific data for managers.

3.2.2 PROJECT DESCRIPTION

Text2SQL agent should produce SQL queries based on the database it works on. It should get a text input from the employee and then feed it into the LLM model. The LLM model should generate the SQL query and get the results from the database by executing the query. The agents should have some specific information about the database, either within its system prompt or within its vector database. Also, for the LLM model, we should use the local LLM model at the insurance company's server in order to prevent any data leak.

3.2.3 TECHNICAL ASPECTS

Firstly, the open source solutions were searched in this field in order to use them or at least get inspired by them. After some research, a python library called Vanna AI was found. Later on similar open source solutions were found, but they were nearly identical to Vanna AI, and Vanna AI had the most extensive documentation compared to others. Vanna AI has a simple working mechanism. It uses an LLM model either locally or via an API key. When an input comes to LLM, it gets information about the database from a vector database. Then by using this info it generates the SQL. After that, it uses this SQL to get results and create graphs. We upload DDLs (Data Definition Language), documentation about the database, and example SQL queries to the vector database in order to generate accurate queries.

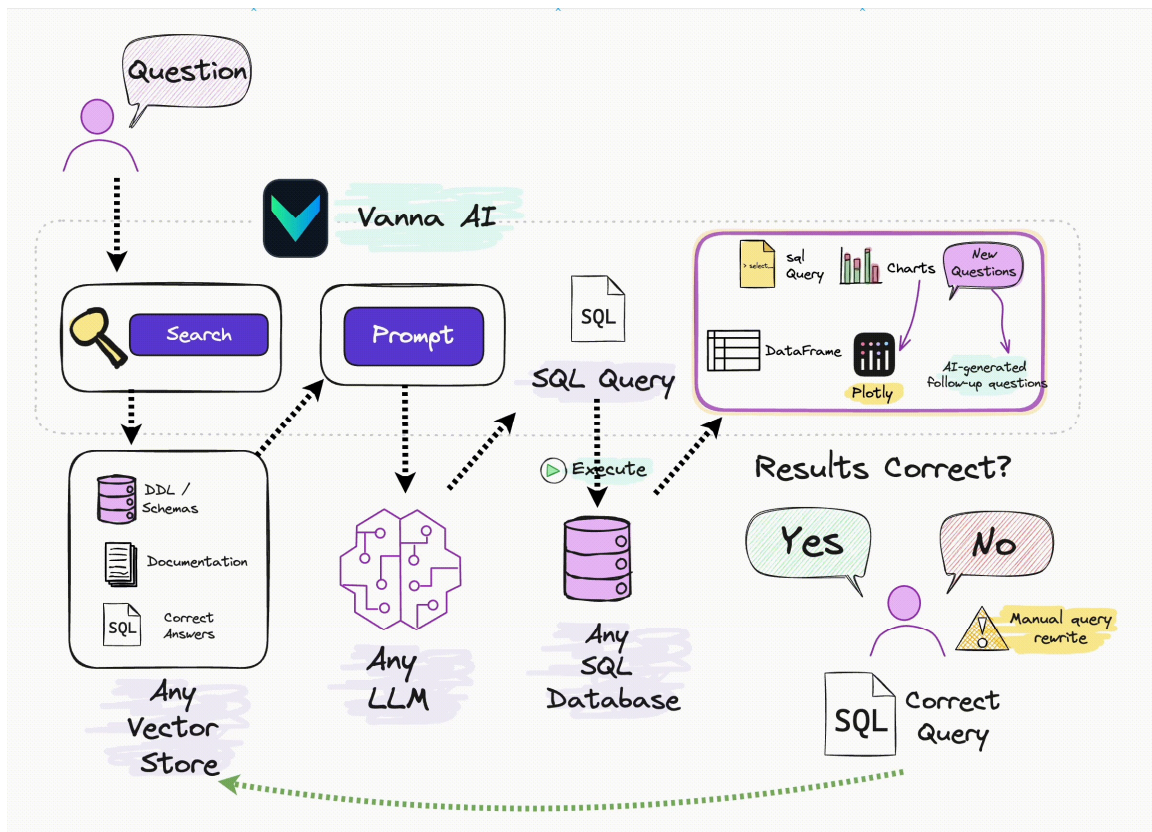


Figure 2: Vanna AI Workflow

As a second solution, another open source model were found with more extensive documentation and youtube videos created by directly its developer. It was a multi agent model which was developed by using Lang Chain for a specific Brazilian Retail Dataset. So the solution was redesigned for our insurance company data set by changing specific nodes, system prompts, and the knowledge base. In this solution, a knowledge base was created by uploading table descriptions and 5 random rows. Then those rows were given to the data annotator agent in order to refine it and create the final version of the database. After that knowledge base is converted to a .pkl file to be used later on. After creating the knowledge base, we try to generate SQL queries. SQL creation multiagent workflow works as following: First, our natural language input is forwarded to a router agent. The router agent analyzes the input and determines which tables are needed for that query. Based on those necessary tables,

it routes the input to the agents that are responsible from those table. We have three tables and three corresponding agents named `ana_veri`, `firma_mapping`, and `urun_mapping`. Those agents check which columns from those tables are needed. Then we input this information to the filter check agent. As its name suggest it check whether any extra filters needs to be done in order to create the requested SQL query. If the answer is yes the send the information to one extra layer called `fuzzy_filter`, which determines which columns need to be filtered. Then all this information is sent to `query_generator`, where an SQL query is created, and finally, the generated query is polished by `query_validation` as a final touch.

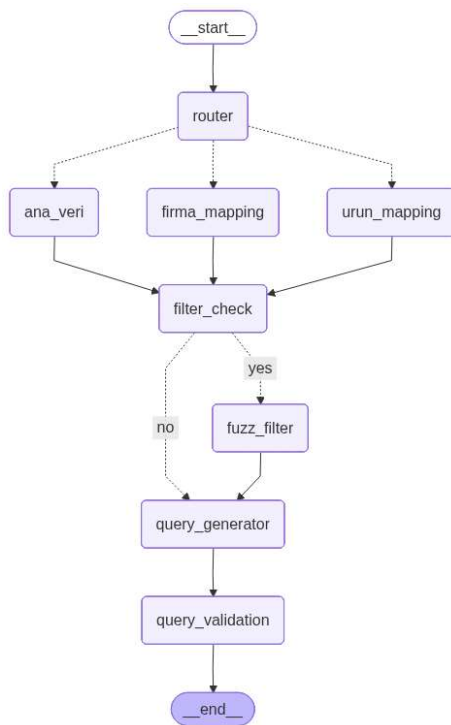


Figure 3: LangChain Node structure

3.2.4 METHODOLOGY and TOOLS

PostgreSQL was used in both solutions because the database was on PostgreSQL.

Also, both solutions employed LLM models for their agents. For the first solution, Vanna AI

library is the main tool, and for the second solution, LangChain was the tool used to develop the multi agent structure.

3.2.5 PROJECT RESULTS

Vanna AI based solution was discarded due to its already predefined structure. We could not easily modify it, and it did not have an extensive documentation explaining how the internal structure works. Documentation was mostly about why to use it and how to use it. Their RAG based structure was discussed a little bit, but that was insufficient. The second solution based on LangChain seemed promising because it could handle basic queries successfully, but unfortunately, when it was tested on more complex queries, it generated overengineered, complex, and false queries. It even sometimes caused error due to some agent outputting invalid formats of text by ignoring their system prompt. The output format issue was solved by modifying the prompt, but the overengineering problem continued to exist. So instead of using this solution, we used another solution that was found by my supervisor in this project.

3.2.6 LITERATURE REVIEW

For this project I have used two main references. Firstly, I used Vanna AI's github link and their read me file (Vanna-Ai, n.d.). Secondly, I used the Langchain project's github link (Applied-Gen-Ai, n.d.) and its youtube video that explains how the system works (AppliedGenAI, 2025).

4. INTERNSHIP EXPERIENCE

4.1 RELATION to UNDERGRADUATE CURRICULLUM

During my internship, I needed to learn all about agentic systems from my own research because unfortunately, there were no courses on agentic systems. However, there is a new course called Large Language Models: Theoretical Foundations and Practical Applications (CS 455), which can help future students to be more informed about this topic. Other than that, there were some courses which was beneficial to me during the internship. For the Text2SQL project, knowing SQL was definitely a plus, so CS 306 was helpful. For understanding agentic systems, having a background in data science and machine learning was also helpful, so we can also count DSA 210 and CS 412.

4.2 DIFFICULTIES

The main difficulty I experienced was that I didn't have a lot of information on how to develop AI agents. So, I needed to read resources to learn more about them. Also, I needed to learn Langflow and LangChain from the start, which also took some time.

4.3 NEW SKILLS

I learned about how to develop AI agents on Langflow and LangChain. Also, I developed my prompt engineering skills in order to affect AI agents' behaviour. Moreover, I learned what RAG is and how I can use it to improve AI agent accuracy and decrease its hallucinations.

5. CONCLUSIONS

I have worked in two main projects in this internship, which are CV Analyzer Agent and Text2SQL Agent. For CV Analyzer Agent, two different methods were tried and the last method I tried was successful. For the Text2SQL Agent, again two different methods were tried but none of them succeeded complete, so another method was employed to solve the

problem. At the end both project were finished at the time of my internship and I got valuable experience for developing agentic solutions.

6. RECOMMENDATIONS

For the students that wants to develop agentic solutions, I will recommend them to take Large Language Models: Theoretical Foundations and Practical Applications (CS 455) which will be helpful them to have some background knowledge about the topic. Also, if there are any students that will make 2 internship at the summer period, I will strongly recommend them to work in a startup company at least in one of those internships to see difference between the startups and big corporations.

7. REFERENCES

- Sudha, G., K, S. K., S, S. J., D, N., S, S., & G, K. T. (2021). Personality Prediction Through CV Analysis using Machine Learning Algorithms for Automated E-Recruitment Process. *2021 4th International Conference on Computing and Communications Technologies (ICCT)*, 617–622. <https://doi.org/10.1109/iccct53315.2021.9711787>
- Singh, N. T., Chanana, A., Jain, D., & Kumar, R. (2023). Personality prediction through CV analysis using machine learning techniques. *ICAECT*, 1–6. <https://doi.org/10.1109/icaect57570.2023.10117883>
- Haddad, R., & Mercier-Laurent, E. (2021). Curriculum Vitae (CVs) evaluation using machine learning approach. In *IFIP advances in information and communication technology* (pp. 48–65). https://doi.org/10.1007/978-3-030-80847-1_4
- Gupta, N. K., Raj, A., Aleem, A., & Jain, A. (2025). Resume Analysis Through Machine Learning. *ICCSAI*, 1305–1308. <https://doi.org/10.1109/iccsai64074.2025.11064067>
- Onukwugha, C., Ofoegbu, C., Aliche, O., & Betrand, C. (2024). Resume optimization model using machine learning techniques. *International Journal of Intelligent Information Systems*, 13(5), 109–116. <https://doi.org/10.11648/j.ijiis.20241305.12>

Bhoir, N., Jakate, M., Lavangare, S., Das, A., & Kolhe, S. (2023). Resume Parser using hybrid approach to enhance the efficiency of Automated Recruitment Processes. *Authorea* (Authorea). <https://doi.org/10.22541/au.168170278.82268853/v1>

Applied-Gen-Ai. (n.d.). *GitHub - applied-gen-ai/txt2sql*. GitHub. <https://github.com/applied-gen-ai/txt2sql/tree/main>

Vanna-Ai. (n.d.). *GitHub - vanna-ai/vanna: Chat with your SQL database. Accurate Text-to-SQL Generation via LLMs using RAG*. GitHub. <https://github.com/vanna-ai/vanna?tab=readme-ov-file>

AppliedGenAI. (2025, June 9). *Text to SQL AI Agent using LangGraph #1 : Introduction* [Video]. YouTube. <https://www.youtube.com/watch?v=7MrQKtGkCMI>